

P2 - Saimazoom

Objetivo

En esta práctica el objetivo es implementar un sistema distribuido sencillo usando RabbitMQ (colas de mensajes). Hay que implementar toda la cadena logística de un marketplace llamado Saimazoom. En este escenario existen los siguientes actores:

- **Cliente**, que realiza y gestiona pedidos de productos.
- **Controlador** central, que gestiona todo el proceso.
- **Robots**, que se encargan de buscar los productos en el almacén y colocarlos en las cintas transportadoras.
- **Repartidores**, encargados de transportar el producto a la casa del cliente.

Para ello, será necesario implementar las operaciones habituales en este tipo de escenarios, que se realizarán en el siguiente orden: creación/consulta/cancelación de un pedido. La cancelación del pedido, solo se permite antes que el pedido haya empezado a ser tramitado por un repartidor. Se entrega un documento de requisitos, a completar, para definir de forma más clara las funcionalidades necesarias.

Para poder llevar a cabo la práctica, deberás comenzar realizando una serie de decisiones de diseño (documentándolas adecuadamente a través de un diagrama de clases, diagrama de estados) que incluirán, como mínimo, las siguientes:

- Diagrama de estados en los que puede estar un pedido en cada momento
- Diagrama de clases
- Formato y sintaxis de los mensajes (RFC)

Consejos de desarrollo

- Hay que entender bien qué es necesario implementar y qué no. Por ejemplo, no hace falta llevar un registro de los productos que hay en el almacén, podemos simular que un robot no encuentra un producto con una probabilidad.
- Para facilitar el desarrollo, puede utilizarse el siguiente truco: crear los distintos actores, en lugar de como programas independientes, como hilos dentro del proceso principal (controlador). En cualquier caso, antes de entregar la práctica debe revertirse el cambio.
- Siempre es buena idea atacar el desarrollo dividiendo el problema en partes, para aislar y reducir las posibles fuentes de errores. Así, para desarrollar y probar el repartidor, por ejemplo, se pueden generar e insertar en la cola mensajes de prueba, de forma que no haga falta que esté ya implementado ningún otro módulo.

Calidad del código

- El código fuente debe estar correctamente comentado
- Se valorará que el sistema se organice como un paquete de Python

- Se deben implementar un número suficiente de tests unitarios.
Idealmente, se recomienda utilizar una metodología de desarrollo TDD (Test-Driven Development)

1. Funcionalidad

1.4. Colas de mensajes

Para la implementación de este sistema es necesario utilizar colas de mensajes; para ello, utilizaremos RabbitMQ: <https://www.rabbitmq.com/> .

Se recomienda leer los tutoriales de <https://www.rabbitmq.com/getstarted.html> , y decidir qué esquemas (work queues, publish/subscribe, ...) son los que tenemos que aplicar en esta práctica.

Para las pruebas se puede lanzar un servidor de RabbitMQ en local con la imagen de docker que ofrece la página (https://registry.hub.docker.com/_/rabbitmq/). Sin embargo, se aconseja para el desarrollo y la entrega usar el servidor está desplegado en redes2.ii.uam.es (cambiar 'localhost' de ConnectionParameters por 'redes2.ii.uam.es')

Para que en el servidor compartido no existan conflictos en los nombres de las colas de mensajes, poned a todas como prefijo GRUPO-PAREJA_ y después el nombre de la cola, por ejemplo: 2321-01_robots.

También, como habrá problemas durante el desarrollo, añadir los siguientes argumentos a `queue_declare`:

- `durable=False` : para que las colas no se guarden tras el reinicio del servidor
- `auto_delete=True` : para que la cola se borre después de que los consumidores se desconecten

Actividad previa

[Examen - Práctica 1 \(2313-2393\)](#)

Ir a...

Siguiente actividad

[Recursos P2](#)